



**DEPARTAMENTO
DE COMPUTACION**

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico Final

System Programming - Kernel RISC-V 32 bits

30 de Febrero de 2026

Arquitectura y organización del computador

Integrante	LU	Correo Electrónico
Aurora Velasco	123/20	velascocata24@gmail.com
Francis Ricle	456/20	francisricleruiz@gmail.com
Mauro Dominguez	789/20	maurod200304@gmail.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - Pabellón I
Intendente Güiraldes 2160 - C1428EGA
Ciudad Autónoma de Buenos Aires - Argentina
Tel/Fax: (54 11) 4576-3359
<http://exactas.uba.ar>

Índice

Introducción	3
Abstract	3
Recursos	3
QEMU	3
Clang/LLVM toolkit	3
OpenSBI	3
Makefile	3
Boot	4
Hello world!	4
Macros	4
Panic	4
Excepciones	4
Memoria	4

Introducción

Abstract

La idea de este trabajo es desarrollar un micro kernel para la arquitectura RISC-V de 32 bits, usando de referencia el micro kernel x86 implementado en la cursada de AyOC y el port de UNIX v6 para RISC-V “xv6”. Buscamos poder implementar un sistema de interrupciones acorde a la arquitectura RISC-V, paginación de memoria, scheduler multitarea, filesystem, además de programas de usuario para probar y un entorno visual con ventanas y manejo de escritorio simple.

Recursos

QEMU

Para la virtualización usamos el modelo de placa ‘virt’ Generic Virtual Platform, que no emula un procesador de hardware real, sino que está pensado para usarse en máquinas virtuales sin las limitaciones específicas que cada productor puede establecer en sus modelos. Decidimos usar la versión genérica porque nos gustaría que este sistema se pueda adaptar a cualquier hardware más fácilmente.

virt supports PCI, virtio, recent CPUs and large amounts of RAM. It also supports 64-bit CPUs.

- Up to 512 generic RV32GC/RV64GC cores, with optional extensions
- Core Local Interruptor (CLINT)
- Platform-Level Interrupt Controller (PLIC)
- CFI parallel NOR flash memory
- 1 NS16550 compatible UART
- 1 Google Goldfish RTC
- 1 SiFive Test device
- 8 virtio-mmio transport devices
- 1 generic PCIe host bridge
- The fw_cfg device that allows a guest to obtain data from QEMU

Clang/LLVM toolkit

Para compilación usamos Clang y LLVM, LLD para linkear, porque estas herramientas están mejor optimizadas para compilar en RISC-V.

Para bootear usamos la dirección de arranque especificada por RISC-V al no estar emulando el hardware de un fabricante específico.

En principio, el mapa de memoria no está definido, pero en la primer parte vamos a usar la UART y luego vamos a extender el kernel para que sea compatible con un teclado y pantalla emulados a través de virtio.

Tomamos la decisión de escribir todo el código que podamos en C y lo que debe ser escrito en assembler hacerlo en un archivo aparte y no hacer inline assembler porque nos parece más legible.

OpenSBI

Makefile

En el makefile hicimos las siguientes reglas:

- debug. Es la regla principal que usamos, esta lanza qemu en background y gdb ademas de conectarse a qemu para debuggear

- run. Esta regla lanza solo qemu sin gdb.
- kernel. Esta es regla que compila el kernel.
- report. Esta regla es para compilar este reporte.

Archivos:

- kernel.c. Archivo principal donde se encuentra kmain nuestra función principal.
- supervisor.S. Aca pondremos las funciones que requieran escribir instrucciones específicas como `ecall` o `csrw`
- libc.c. Esta es muestra “stdlib”.
- kernel.ld. Layout de la memoria.
- opensbi.bin. Nuestro firmware por defecto qemu utiliza este firmware si no se le especifica. Pero nosotros decidimos tener el archivo.

Configuración de gdb: Agregamos comandos a gdb para facilitarnos el desarrollo:

- `kernel_reload`. Vuelve a compilar el kernel, resetea qemu y vuelve a cargar los símbolos a gdb.
- `kernel_exit`. Para salir de manera segura. Si cerramos qemu al estar corriendo el background no podemos volver a lanzarlo

Boot

El primer paso es definir el layout de memoria en el archivo `kernel.ld`. Fijamos el inicio del kernel en la posición de memoria `0x80200000` ya que en la posición `0x80000000` está ubicado el OpenSBI (el firmware estándar de RISC-V). Luego, alineamos cada sección de memoria (rodata, data, bss, stack) a 4 bytes y usamos la instrucción `KEEP` para decirle al compilador que el `.text.boot` no es dead code. Por último saltamos los primeros 128 kb, reservados para el stack del kernel.

Hello world!

El kernel arranca en la dirección definida en los pasos de arriba, en este caso es la tag ‘boot’ del archivo `kernel.S`, donde se define el stack pointer en la dirección dada por `kernel.ld`, luego se salta a `kmain` en `kernel.c`. Para escribir en pantalla se usa `kputs` de `libc.c`, que por cada carácter llama a `kputc`, que llama a OpenSBI para imprimir el carácter por el puerto UART.

QEMU se encarga de enviar la salida del puerto UART a una ventana gráfica de SDL. En este caso lo configuramos así para tener por un lado en la terminal gdb corriendo y por otro tener el display de lo que sucede en el kernel. Otra manera hubiese sido que QEMU mande la salida del UART directamente al stout.

Luego de imprimir en pantalla la función `halt` espera por interrupciones y sino hace un loop infinito.

Macros

Panic

Excepciones

Memoria